

Part-of-Speech Tagging using Hidden Markov Models (HMMs)

1. Assignment Overview

- **Course:** Natural Language Processing S2-25_AIMLCZG530
- **Task:** Implement a Hidden Markov Model (HMM) from scratch for Part-of-Speech (POS) tagging, use the Viterbi algorithm for decoding, and critically evaluate its performance against a pre-trained POS tagger.
- **Dataset:** NLTK Brown corpus (Use Training Category: **news** Testing Category: **fiction**)
- **Note:** 1 Mark is given for use of BITS Lab.

2. Learning Objectives

By completing this assignment, students will demonstrate the ability to:

1. **Formulate and implement** the probabilistic framework of HMMs (Transition and Emission matrices) from text data.
2. **Design and code** the dynamic programming-based Viterbi algorithm for sequence labelling.
3. **Address real-world NLP constraints**, such as the "unknown words" problem, using smoothing techniques or back-off models.
4. **Critically evaluate** machine learning models using standard sequence labelling metrics and perform error analysis against established baselines.

Part 1: Data Preprocessing & Exploratory Data Analysis (EDA) [2 Marks]

The objective of this part is to clean, analyze, and format a tagged corpus to extract the transition and emission states.

- a. Data Loading
- b. Exploratory Data Analysis (EDA)
- c. Vocabulary Handling & Unknown Words
- d. Sentence Padding
- e. Data Splitting

Part 2: Model Architecture & Parameter Estimation [4 Marks]

Build a fundamental Bigram Hidden Markov Model from scratch. You must calculate the probability matrices explicitly using maximum likelihood estimation (MLE) from the training set.

- Estimate the **Transition Probability Matrix (A)** and **Emission Probability Matrix (B)** from the training data using Maximum Likelihood Estimation (MLE).
- Implement a handling mechanism for out-of-vocabulary (OOV) words
- Implement the Viterbi algorithm from scratch to find the most likely sequence of POS tags for the test sentences. Your algorithm must take a sequence of words as input and output the most likely sequence of hidden POS tags.
- Avoid using high-level wrapper libraries (like `nltk.tag.hmm` or `sklearn`) for the core implementation. Vectorized implementations (e.g., using NumPy) are highly encouraged for computational efficiency.

Part 3: Inference and Evaluation [4 Marks]

- a) Sequence Inference (POS Tagging): Use your trained HMM and Viterbi decoder to predict the POS tag sequences for any 4 test sentences. Pick sentences 2 from training dataset and 2 from test dataset.
- b) Identify and print 3 distinct sentences where the Viterbi decoder misclassified at least one tag. Briefly analyze why the model struggled with these specific words (e.g. lexical ambiguity, rare transitions).
- c) Run your Viterbi decoder across the entire 20% Test Set. Calculate and report:
 - a) Overall token-level accuracy.
 - b) A confusion matrix for the top 5 most common POS tags.

Part D: Baseline Comparison [1 Mark]

- Load a pre-trained, state-of-the-art POS tagger (e.g., SpaCy's tagger, Stanza, or NLTK's `PerceptronTagger`).
- Run the pre-trained model on the same test set. Discuss why your HMM failed but the pre-trained model succeeded.

2. Deliverables

- Jupyter notebook with clear mention of all the members of your team including Name and Bits ID.
- The notebook must execute from top to bottom without errors (Kernel -> Restart & Run All must pass successfully). No cells should display execution errors.
- Include explicit code output cells (or cleanly formatted markdown blocks immediately following the code) showing:
 - The EDA POS tag distribution bar chart.
 - A heatmap or structured DataFrame snapshot of a portion of your Transition Matrix.
 - The decoded POS tag outputs for Test Sentences.
 - The final Evaluation Accuracy and Confusion Matrix.
- Screenshots with test cases showing the inferences and predictions
- Screenshots showing use of BITS CSIS Labs.
- Any queries related tot his assignment should be addressed to Course LE, Ajay Naik